

APOSTILA COMPLETA: Arquitetura de Software para E-commerce Guia Definitivo do Desenvolvedor Profissional

ÍNDICE Introdução

Fundamentos do Desenvolvimento Profissional

Arquitetura de Software

Estrutura de Projetos

Padrões de Design

Testes e Qualidade

DevOps e CI/CD

Documentação

Passo a Passo Prático

Checklists e Templates

1. INTRODUÇÃO {#introducao} 1.1 Sobre esta Apostila Esta apostila foi criada para ser seu guia definitivo para desenvolver aplicações profissionais, utilizando as melhores práticas do mercado, independentemente da linguagem de programação.

1.2 Objetivos ☒ Ensinar arquitetura de software do zero

☒ Fornecer templates reutilizáveis

☒ Demonstrar padrões de projeto na prática

☒ Preparar para desafios reais do mercado

1.3 Público-Alvo Desenvolvedores iniciantes a intermediários

Profissionais buscando padronização

Equipes que desejam adotar boas práticas

2. FUNDAMENTOS DO DESENVOLVIMENTO PROFISSIONAL {#fundamentos} 2.1 Os Pilares do Desenvolvimento de Elite

PILARES FUNDAMENTAIS:  SOLID: - S: Single Responsibility (Uma classe, uma responsabilidade) - O: Open/Closed (Aberto para extensão, fechado para modificação) - L: Liskov Substitution (Subtipos devem ser substituíveis) - I: Interface Segregation (Interfaces específicas) - D: Dependency Inversion (Dependa de abstrações, não de concretos)

 DRY: - Don't Repeat Yourself (Não repita código) - Cada conhecimento deve ter uma representação única

 KISS: - Keep It Simple, Stupid (Mantenha simples) - Simplicidade sobre complexidade

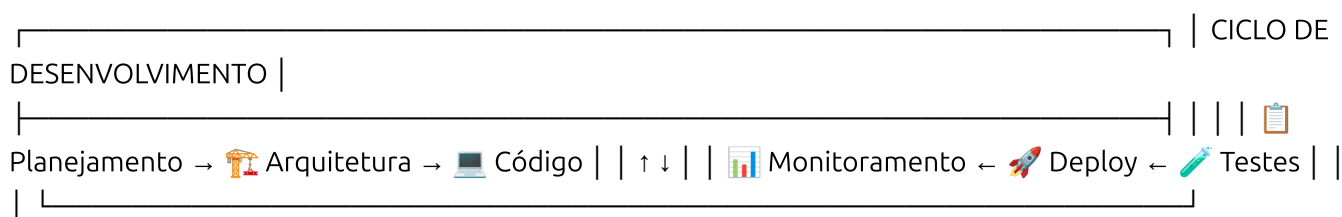
📌 YAGNI: - You Aren't Gonna Need It (Você não vai precisar disso) - Não implemente o que não é necessário agora

📌 DDD: - Domain-Driven Design (Design orientado ao domínio) - Foco no negócio e na linguagem ubíqua

2.2 Mentalidade do Desenvolvedor 10x

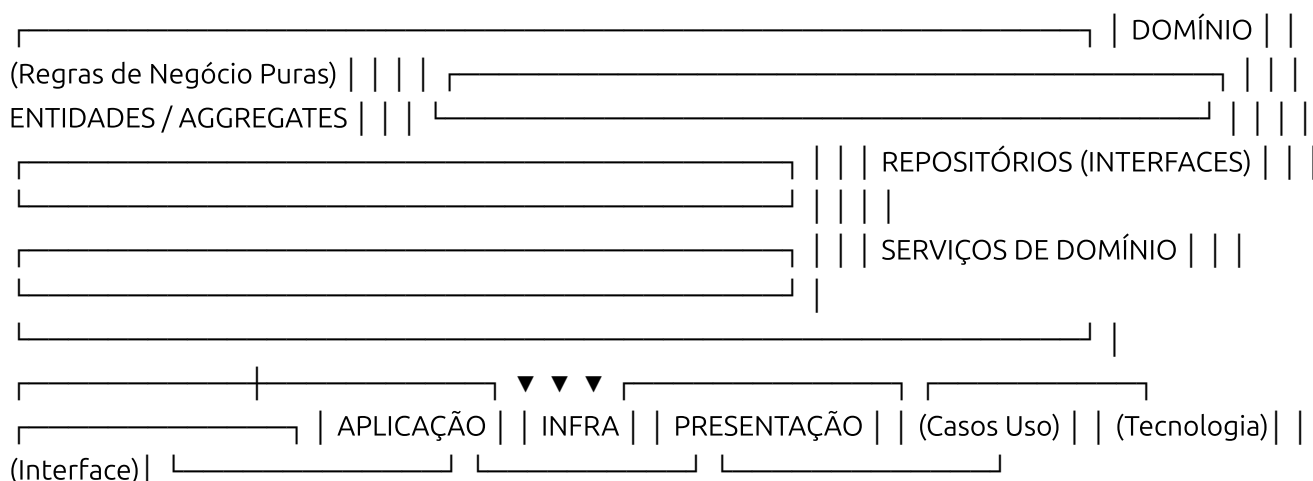
🎯 CARACTERÍSTICAS DE UM DEV 10x: ☐ Pensa em arquitetura antes de codificar ☐ Escreve testes automaticamente ☐ Documenta decisões importantes ☐ Refatora constantemente ☐ Ensina e compartilha conhecimento ☐ Pensa em escalabilidade ☐ Considera segurança desde o início ☐ Automatiza tarefas repetitivas ☐ Mantém-se atualizado ☐ Contribui com a comunidade

2.3 O Ciclo de Desenvolvimento Profissional



3. ARQUITETURA DE SOFTWARE {#arquitetura} 3.1 Arquitetura Hexagonal (Ports & Adapters) A

Arquitetura Hexagonal é o padrão mais utilizado por grandes empresas.



3.2 Componentes da Arquitetura 3.2.1 Domain Layer (Camada de Domínio) Responsabilidade: Regras de negócio puras, independentes de tecnologia.

```
price = $price; $this->stock = $stock; } public function reduceStock(int $quantity): void { if (!$this->stock->hasEnough($quantity)) { throw new InsufficientStockException(); } $this->stock->reduce($quantity); } } // Domain/ValueObjects/Money.php namespace Domain\ValueObjects; class Money { public function __construct( private readonly float $amount, private readonly string $currency = 'BRL' ) { if ($amount < 0) { throw new InvalidMoneyException(); } } public function add(Money $other): Money { if ($this->currency !== $other->currency) { throw new CurrencyMismatchException(); } return new Money( $this->amount + $other->amount, $this->currency ); } } 3.2.2 Application Layer (Camada de Aplicação) Responsabilidade: Orquestrar casos de uso, coordenar o domínio. name, new Money($command->price), new Stock($command->stock) ); // 2. Salvar $saved = $this->repository->save($product); // 3. Disparar eventos $this->dispatcher->dispatch(new ProductCreatedEvent($saved)); // 4. Retornar DTO return ProductDTO::fromEntity($saved); } } 3.2.3 Infrastructure Layer (Camada de Infraestrutura) Responsabilidade: Implementações concretas de tecnologia (banco, cache, filas). $product->id, [ 'name' => $product->name, 'price' => $product->price->amount(), 'stock'
```

```
=> $product->stock->quantity(), 'currency' => $product->price->currency() ] ); return $product->withId($model->id); } public function findById(int $id): ?Product { $model = EloquentProduct::find($id); if (!$model) { return null; } return new Product( $model->name, new Money($model->price, $model->currency), new Stock($model->stock) )->withId($model->id); } }
```

3.2.4 Presentation Layer (Camada de Apresentação)

Responsabilidade: Interface com o usuário (API, Web, CLI).
, price: (float) , stock: (int) , description:); \$product = \$this->handler->handle(\$command); return response()->json(ProductResource::make(\$product), 201); }

3.3 Comparação de Arquiteturas

Aspecto	Hexagonal MVC	Clean Architecture	Independência	Alta	Baixa	Alta
Testabilidade	Excelente	Limitada	Excelente	Média	Baixa	Alta
Complexidade	Média	Baixa	Alta	Curva de Aprendizagem	Média	
Manutenibilidade	Excelente	Média	Excelente	Popularidade	Alta	Muito Alta

4. ESTRUTURA DE PROJETOS {#estrutura}

4.1 Estrutura Universal (Multi-Linguagem)

```

project-name/ |
├── src/ # Código fonte
├── Domain/ # Camada de Domínio
├── Entities/ # Entidades de negócio
├── ValueObjects/ # Objetos de valor
├── Aggregates/ # Agregados
├── Events/ # Eventos de domínio
├── Exceptions/ # Exceções de domínio
├── Repositories/ # Interfaces de repositórios
├── Application/ # Camada de Aplicação
├── Commands/ # Comandos (CQRS)
├── Queries/ # Consultas (CQRS)
├── Handlers/ # Manipuladores
├── DTOs/ # Data Transfer Objects
├── Interfaces/ # Interfaces de serviço
├── Services/ # Serviços de aplicação
├── Infrastructure/ # Camada de Infraestrutura
├── Database/ # Configuração DB
├── Repositories/ # Implementações
├── Cache/ # Cache
├── Queue/ # Filas
├── Services/ # Serviços externos
├── Providers/ # Injeção de dependência
├── Presentation/ # Camada de Apresentação
├── Controllers/ # Controladores
├── Middleware/ # Middleware
├── Requests/ # Validações
├── Resources/ # Transformadores
├── Views/ # Templates
├── tests/ # Testes
├── Unit/ # Testes Unitários
├── Integration/ # Testes de Integração
├── Feature/ # Testes Funcionais
├── E2E/ # Testes End-to-End
├── docs/ # Documentação
├── api/ # API Docs
├── architecture/ # ADR
├── guides/ # Guias
├── scripts/ # Scripts de automação
├── docker/ # Docker configurações
├── .github/workflows/ # CI/CD
├── .env.example # Variáveis de ambiente
├── docker-compose.yml # Docker Compose
├── Dockerfile # Dockerfile
├── Makefile # Comandos automáticos
├── README.md # Documentação inicial
├── CHANGELOG.md # Histórico de mudanças
├── LICENSE # Licença

```

4.2 Estrutura Laravel (Seu Projeto Atual)

```

app/
├── Actions/ # → Application/Commands
├── Admin/Produto/
├── AjustarEstoqueAction.php
├── AtualizarProdutoAction.php
├── CriarProdutoAction.php
├── DeletarProdutoAction.php
├── DTOs/ # → Application/DTOs
├── Requests/
├── CreateProductRequestDTO.php
├── UpdateProductRequestDTO.php
├── Responses/
├── ApiResponseDTO.php
├── ProductResponseDTO.php
├── PaymentResponseDTO.php
├── Models/ # → Domain/Entities
├── Produto.php
├── Pedido.php
├── User.php
├── Banner.php
├── Repositories/ # → Infrastructure/Repositories
├── Contracts/ # → Domain/Repositories
├── ProdutoRepositoryInterface.php
├── UserRepositoryInterface.php
├── ProdutoRepository.php
├── Services/ # → Application/Services
├── Contracts/ # → Application/Interfaces
├── ProductServiceInterface.php
├── PaymentServiceInterface.php
├── ProductService.php
├── Http/Controllers/ # → Presentation/Controllers
├── Admin/
├── Api/
├── Auth/
├── Http/Requests/ # → Presentation/Requests
├── Http/Resources/ # → Presentation/Resources
├── Http/Middleware/ # → Presentation/Middleware

```

5. PADRÕES DE DESIGN {#padroes}

5.1 Repository Pattern

Propósito: Abstrair a camada de dados.

```

model->find($id); return $record ? $this->toDomain($record) : null; } public function save(Product $product): Product { $record = $this->model->updateOrCreate( ['id' => $product->id], $this->toPersistence($product) ); return $this->toDomain($record); } private function toDomain(ProductModel

```

```

$record): Product { return new Product( $record->id, $record->name, new Money($record->price), new
Stock($record->stock) ); } private function toPersistence(Product $product): array { return [ 'name' =>
$product->name, 'price' => $product->price->amount(), 'stock' => $product->stock->quantity(), ]; } } 5.2
Service Pattern Propósito: Encapsular lógica de negócio complexa. validateProductData($dto); // 2. Criar
entidade $product = new Product( $dto->name, new Money($dto->price), new Stock($dto->stock) ); // 3.
Salvar $saved = $this->repository->save($product); // 4. Disparar eventos $this->dispatcher->dispatch( new
ProductCreatedEvent($saved) ); // 5. Retornar DTO return ProductDTO::fromEntity($saved); } public function
adjustStock(int $productId, int $quantity): void { $product = $this->repository->find($productId); if
(!$product) { throw new ProductNotFoundException(); } $this->stockService->adjust($product, $quantity);
$this->repository->save($product); } } 5.3 DTO Pattern Propósito: Transferir dados entre camadas. id, name:
$product->name, price: $product->price->amount(), stock: $product->stock->quantity(), description:
$product->description, createdAt: $product->createdAt?->format('Y-m-d H:i:s') ); } public function toArray():
array { return [ 'id' => $this->id, 'name' => $this->name, 'price' => $this->price, 'stock' => $this->stock,
'description' => $this->description, 'created_at' => $this->createdAt ]; } } 5.4 Factory Pattern Propósito: Criar
objetos complexos. reduceStock(3); // Assert $this->assertEquals(7, $product->stock->quantity()); } /** @test
*/ public function it_throws_exception_when_insufficient_stock() { $this-
>expectException(InsufficientStockException::class); $product = new Product( 'Test Product', new
Money(100.00), new Stock(5) ); $product->reduceStock(10); } } 6.3 Teste de Integração repository =
app(ProductRepositoryInterface::class); /** @test */ public function it_can_save_and_find_product() { //
Arrange $product = new Product( 'Test Product', new Money(100.00), new Stock(10) ); // Act $saved = $this-
>repository->save($product); $found = $this->repository->find($saved->id); // Assert $this-
>assertEquals($saved->id, $found->id); $this->assertEquals('Test Product', $found->name); $this-
>assertEquals(100.00, $found->price->amount()); } } 6.4 Teste Funcional (Feature) admin = User::factory()-
>admin()->create(); /** @test */ public function admin_can_create_product() { // Arrange $this-
>actingAs($this->admin); $data = [ 'name' => 'New Product', 'price' => 199.90, 'stock' => 20 ]; // Act $response
= $this->postJson('/api/products', $data); // Assert $response->assertStatus(201) -
>assertJsonPath('data.name', 'New Product'); $this->assertDatabaseHas('products', [ 'name' => 'New
Product', 'price' => 199.90 ]; } } 7. DEVOPS E CI/CD {#devops} 7.1 Docker Setup Completo # Dockerfile (Multi-
stage) FROM php:8.2-fpm AS builder WORKDIR /var/www/html # Install dependencies RUN apt-get update
&& apt-get install -y \ git curl libpng-dev libonig-dev libxml2-dev zip unzip \ && docker-php-ext-install
pdo_mysql mbstring exif pcntl bcmath gd # Install Composer COPY --from=composer:latest
/usr/bin/composer /usr/bin/composer # Copy application COPY . . # Install dependencies (production) RUN
composer install --no-interaction --optimize-autoloader --no-dev # Stage 2: Nginx FROM nginx:alpine COPY --
from=builder /var/www/html /var/www/html COPY docker/nginx/conf.d /etc/nginx/conf.d EXPOSE 80 CMD
["nginx", "-g", "daemon off;"] 7.2 GitHub Actions CI/CD # .github/workflows/deploy.yml name: Deploy to
Production on: push: branches: [ main ] jobs: deploy: runs-on: ubuntu-latest steps: - uses:
actions/checkout@v3 - name: Setup PHP uses: shivammathur/setup-php@v2 with: php-version: '8.2'
extensions: mbstring, intl, pdo_mysql - name: Install Dependencies run: composer install --no-interaction --
optimize-autoloader - name: Run Tests run: | php artisan migrate --force php artisan test - name: Deploy to
Production env: DEPLOY_KEY: ${ secrets.DEPLOY_KEY } DEPLOY_HOST: ${ secrets.DEPLOY_HOST } run: |
echo "$DEPLOY_KEY" > deploy_key chmod 600 deploy_key ssh -i deploy_key $DEPLOY_HOST "cd
/var/www/app && git pull && composer install && php artisan migrate --force" 8. DOCUMENTAÇÃO
{#documentacao} 8.1 README.md Template # 🚀 Nome do Projeto > Breve descrição do projeto ## 📋
Índice - [Sobre](#sobre) - [Tecnologias](#tecnologias) - [Pré-requisitos](#pre-requisitos) - [Instalação]
(#instalação) - [Configuração](#configuração) - [Uso](#uso) - [Testes](#testes) - [Deploy](#deploy) -
[Contribuição](#contribuição) - [Licença](#licença) ## 📖 Sobre Descrição detalhada do projeto. ## 🛠️

```

Tecnologias - ****Backend:**** PHP 8.2, Laravel 10 - ****Frontend:**** Vue.js 3, TailwindCSS - ****Database:**** MySQL 8.0, Redis - ****DevOps:**** Docker, GitHub Actions ## 📋 Pré-requisitos - PHP 8.2+ - Composer - Node.js 18+ - NPM/Yarn - MySQL 8.0+ - Docker (opcional) ## ⚙️ Instalação `\\`\\`bash # Clone o repositório git clone https://github.com/yourusername/project.git cd project # Instale as dependências PHP composer install # Instale as dependências Node npm install # Configure o ambiente cp .env.example .env # Gere a chave php artisan key:generate # Execute as migrações php artisan migrate --seed # Inicie o servidor php artisan serve \\`\\` ## 🧪 Testes \\`\\`bash # Execute todos os testes php artisan test # Execute testes unitários php artisan test --testsuite=Unit # Execute testes de integração php artisan test --testsuite=Integration # Execute testes funcionais php artisan test --testsuite=Feature \\`\\` ## 📄 Licença Este projeto está sob a licença MIT.`

8.2 Architecture Decision Record (ADR) Template

ADR-001: Nome da Decisão ## Status [Proposto | Aceito | Rejeitado | Depreciado] ## Contexto Descrição do contexto e problema. ## Decisão A decisão tomada e o racional por trás dela. ## Consequências ### Positivas - Ponto positivo 1 - Ponto positivo 2 ### Negativas - Ponto negativo 1 - Ponto negativo 2 ## Alternativas Consideradas - Alternativa 1: Motivo da rejeição - Alternativa 2: Motivo da rejeição ## Referências - Link para documentação - Link para discussões

9. PASSO A PASSO PRÁTICO {#pratico}

9.1 Criando um Projeto do Zero

Passo 1: Planejamento 📋

PLANEJAMENTO DO PROJETO

- **Definir o Escopo****
 - Quais funcionalidades principais?
 - Quem será o usuário?
 - Qual o objetivo de negócio?
- **Escolher Tecnologias****
 - Backend: Laravel, Node.js, Python?
 - Frontend: React, Vue, Angular?
 - Database: MySQL, PostgreSQL, MongoDB?
 - Cache: Redis, Memcached?
 - Queue: RabbitMQ, Redis?
- **Definir Arquitetura****
 - Hexagonal, Clean, MVC?
 - Padrões de design?
 - Estrutura de pastas?

Passo 2: Setup Inicial

1. Criar estrutura de pastas `mkdir meu-ecommerce cd meu-ecommerce`
2. Inicializar Git `git init git checkout -b develop git branch main`
3. Criar estrutura base `mkdir -p src/{Domain,Application,Infrastructure,Presentation,Shared} mkdir -p tests/{Unit,Integration,Feature,E2E} mkdir -p docs/{api,architecture,guides} mkdir -p docker/{mysql,nginx,redis} mkdir -p .github/workflows`
4. Criar arquivos base `touch README.md touch .gitignore touch docker-compose.yml touch Dockerfile touch Makefile touch .env.example touch phpunit.xml`

Passo 3: Configurar Dependências

- Laravel** `composer create-project laravel/laravel .`
- Node.js** `npm init -y npm install express mongoose dotenv`
- Python** `python -m venv venv source venv/bin/activate pip install fastapi sqlalchemy alembic`

Passo 4: Implementar Camadas //

- 1. Domain Layer**
 - `src/Domain/Entities/Product.php`
 - `src/Domain/ValueObjects/Money.php`
 - `src/Domain/Repositories/ProductRepositoryInterface.php`
- 2. Application Layer**
 - `src/Application/Commands/CreateProductCommand.php`
 - `src/Application/Handlers/CreateProductHandler.php`
 - `src/Application/DTOs/ProductDTO.php`
- 3. Infrastructure Layer**
 - `src/Infrastructure/Repositories/ProductRepository.php`
 - `src/Infrastructure/Database/Migrations/...`
- 4. Presentation Layer**
 - `src/Presentation/Controllers/ProductController.php`
 - `src/Presentation/Requests/CreateProductRequest.php`

Passo 5: Configurar Testes

- Laravel** `php artisan make:test ProductServiceTest --unit php artisan make:test ProductControllerTest`
- Node.js** `npm install --save-dev jest supertest`
- Python** `pip install pytest pytest-asyncio httpx`

9.2 Comandos Úteis (Makefile)

Makefile

```
.PHONY: help install test migrate seed dev build help: @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) | sort | awk 'BEGIN {FS = ":.*?## "}; {printf "\033[36m%-30s\033[0m %s\n", $$1, $$2}'
```

install: ## Instalar dependências `composer install npm install`

migrate: ## Executar migrações `php artisan migrate`

seed: ## Executar seeders `php artisan db:seed`

test: ## Executar testes `php artisan test`

dev: ## Iniciar servidor de desenvolvimento `php artisan serve`

build: ## Build para produção `composer install --no-dev --optimize-autoloader npm run build`

docker-up: ## Iniciar containers Docker `docker-compose up -d`

docker-down: ## Parar containers Docker `docker-compose down`

logs: ## Ver logs `tail -f storage/logs/laravel.log`

9.3 Template de Configuração (.env)

```
# .env.example APP_NAME=MeuEcommerce APP_ENV=local APP_KEY= APP_DEBUG=true APP_URL=http://localhost:8000 DB_CONNECTION=mysql
```

DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=ecommerce DB_USERNAME=root DB_PASSWORD=

CACHE_DRIVER=redis SESSION_DRIVER=redis QUEUE_CONNECTION=redis REDIS_HOST=127.0.0.1

REDIS_PASSWORD=null REDIS_PORT=6379 # APIs MERCADOPAGO_PUBLIC_KEY=

MERCADOPAGO_SECRET_KEY= # Email MAIL_MAILER=smtp MAIL_HOST=smtp.mailtrap.io

MAIL_PORT=2525 MAIL_USERNAME=null MAIL_PASSWORD=null 10. CHECKLISTS E TEMPLATES {#checklists}

10.1 Checklist de Qualidade  CHECKLIST DE QUALIDADE DO PROJETO  CHECKLIST DE QUALIDADE DO PROJETO ## Arquitetura ☐ Arquitetura definida e documentada ☐ Separação de camadas (Domain, Application, Infrastructure, Presentation) ☐ SOLID aplicado ☐ Design Patterns apropriados ## Código ☐ Coding Standards definidos ☐ Code Review implementado ☐ Nomenclatura clara ☐ Comentários úteis ☐ Sem duplicação de código (DRY) ## Testes ☐ Testes Unitários (Cobertura > 80%) ☐ Testes de Integração ☐ Testes Funcionais ☐ Testes E2E ☐ CI/CD com testes automáticos ## Documentação ☐ README.md completo ☐ Documentação da API ☐ ADR (Architecture Decision Records) ☐ Guia de contribuição ☐ CHANGELOG.md ## DevOps ☐ Docker configurado ☐ CI/CD implementado ☐ Logging implementado ☐ Monitoring configurado ☐ Backup strategy definida ## Segurança ☐ CORS configurado ☐ CSRF protection ☐ XSS prevention ☐ SQL Injection prevention ☐ Rate Limiting ☐ Authentication/Authorization ☐ Environment variables seguras ☐ Dependências atualizadas ## Performance ☐ Cache implementado ☐ Queue para tarefas pesadas ☐ Database indexes otimizados ☐ Assets otimizados ☐ Lazy loading implementado ☐ Paginação implementada 10.2 Template de Pull Request ##  Descrição - O que foi feito? - Por que foi feito? - Como foi feito? ##  Testes - [] Testes unitários - [] Testes de integração - [] Testes manuais ##  Checklist - [] Código segue padrões do projeto - [] Documentação atualizada - [] Testes passando - [] Sem warnings - [] Sem breaking changes - [] Performance verificada ##  Screenshots (se aplicável) (Coloque screenshots aqui) ##  Links Relacionados - [Link para ticket] - [Link para documentação] 10.3 Template de API Response { "success": true, "data": { "id": 1, "name": "Produto Exemplo", "price": 99.90, "stock": 20 }, "message": "Produto criado com sucesso", "errors": null, "meta": { "timestamp": "2024-01-01T00:00:00.000Z", "request_id": "abc-123" } } 10.4 Template de Exception Handler validationError(\$e); if (\$e instanceof DomainException) { return \$this->domainError(\$e); } if (\$e instanceof ModelNotFoundException) { return \$this->notFoundError(\$e); } if (\$e instanceof AuthenticationException) { return \$this->unauthorizedError(\$e); } return \$this->genericError(\$e); } private function validationError(ValidationException \$e): JsonResponse { return response()->json(['success' => false, 'message' => 'Validation error', 'errors' => \$e->errors(),], 422); } private function domainError(DomainException \$e): JsonResponse { return response()->json(['success' => false, 'message' => \$e->getMessage(),], 400); } } 11. APRENDENDO MAIS 11.1 Recursos Recomendados  Livros Domain-Driven Design - Eric Evans Clean Architecture - Robert C. Martin Implementing DDD - Vaughn Vernon Design Patterns - Gang of Four Refactoring - Martin Fowler  Cursos Clean Architecture (Pluralsight) Domain-Driven Design (Udemy) Test-Driven Development (Laracasts) Docker Mastery (Udemy)  Podcasts Software Engineering Daily Code with Jason Laravel News Podcast The Changelog  Blogs Martin Fowler Blog David Heinemeier Hansson (DHH) Kent Beck Laravel News 11.2 Comunidades GitHub Stack Overflow Reddit (r/laravel, r/programming) Discord/Slack groups Meetups locais 11.3 Ferramentas Essenciais EDITORES: - VS Code (com extensões) - PHPStorm - Sublime Text VERSIONAMENTO: - Git - GitHub/GitLab/Bitbucket TESTES: - PHPUnit/Pest - Jest/Mocha - Pytest CI/CD: - GitHub Actions - GitLab CI - Jenkins CONTAINERS: - Docker - Docker Compose - Kubernetes MONITORING: - Sentry - New Relic - Datadog 12. EXERCÍCIOS PRÁTICOS 12.1 Exercício 1: Criar um CRUD Completo Desafio: Implementar um CRUD de produtos usando a arquitetura hexagonal. Tarefas: Criar a entidade Product (Domain) Criar o repositório (Infrastructure) Criar o service (Application) Criar o controller (Presentation) Escrever testes para cada camada 12.2 Exercício 2: Implementar Pagamento Desafio: Implementar sistema de pagamento com múltiplos gateways. Tarefas: Criar interface PaymentGateway (Domain) Implementar MercadoPagoGateway (Infrastructure) Implementar PayPalGateway (Infrastructure) Criar PaymentService (Application) Criar PaymentController (Presentation)

12.3 Exercício 3: Sistema de Notificações Desafio: Implementar sistema de notificações assíncronas. Tarefas: Criar evento OrderPlaced (Domain) Criar listeners para email e SMS Configurar queue (Infrastructure) Criar testes de integração 12.4 Exercício 4: Refatoração Desafio: Refatorar um projeto existente. Tarefas:

Identificar violações de SOLID Extrair interfaces Separar camadas Escrever testes Documentar mudanças 13.

CONCLUSÃO 13.1 Pontos-Chave Arquitetura é fundamental - Defina antes de codificar Testes são essenciais -

Escreva testes automaticamente Documentação é crucial - Documente decisões DevOps é necessário -

Automatize o deploy Aprendizado contínuo - Nunca pare de estudar 13.2 Próximos Passos ☒ Escolha um

projeto para praticar ☒ Aplique a arquitetura aprendida ☒ Escreva testes para tudo ☒ Configure CI/CD ☒

Documente cada decisão ☒ Compartilhe seu conhecimento 13.3 Mensagem Final "O conhecimento é a

única coisa que cresce quando é compartilhado." — Osho Continue estudando, praticando e compartilhando!

ANEXOS Anexo A: Referências Rápidas Princípios SOLID Cheat Sheet S - Single Responsibility "Uma classe

deve ter um, e somente um, motivo para mudar." O - Open/Closed "Entidades devem estar abertas para

extensão, fechadas para modificação." L - Liskov Substitution "Subtipos devem ser substituíveis por seus

tipos base." I - Interface Segregation "Muitas interfaces específicas são melhores do que uma interface

geral." D - Dependency Inversion "Dependa de abstrações, não de implementações concretas." Padrões de

Design Cheat Sheet CRIACIONAIS: - Factory: Cria objetos sem expor lógica - Singleton: Uma única instância -

Builder: Constrói objetos complexos ESTRUTURAIS: - Adapter: Converte interfaces - Decorator: Adiciona

funcionalidades - Facade: Simplifica interfaces COMPORTAMENTAIS: - Strategy: Algoritmos intercambiáveis -

Observer: Notifica mudanças - Command: Encapsula requisições Anexo B: Comandos Úteis # Git git init git

add . git commit -m "message" git branch git checkout -b feature git merge git rebase # Docker docker build -

t app . docker-compose up -d docker-compose down docker logs app docker exec -it app bash # Laravel php

artisan make:model Product -m php artisan make:controller ProductController --resource php artisan

make:test ProductTest --unit php artisan migrate php artisan tinker php artisan optimize # NPM npm init -y

npm install package-name npm run dev npm run build # Composer composer require package-name

composer install composer update composer dump-autoload Você é um arquiteto de software especialista

em desenvolvimento de sistemas escaláveis, com mais de 15 anos de experiência em grandes empresas de

tecnologia. Sua especialidade é criar projetos com arquitetura hexagonal (ports & adapters), aplicando DDD,

SOLID, TDD e as melhores práticas do mercado. BASEADO NA CONVERSA ANTERIOR SOBRE ARQUITETURA

DE SOFTWARE, CRIE: ## 1. ESTRUTURA DE PASTAS COMPLETA Crie a estrutura de pastas completa para um

projeto [NOME DO PROJETO] usando a arquitetura hexagonal, incluindo: #### Domain Layer - Entities

(entidades de negócio) - Value Objects (objetos de valor) - Aggregates (agregados) - Domain Events (eventos

de domínio) - Domain Exceptions (exceções de domínio) - Repository Interfaces (interfaces de repositório) -

Domain Services (serviços de domínio) #### Application Layer - Commands (CQRS - comandos) - Queries

(CQRS - consultas) - Handlers (manipuladores) - DTOs (Data Transfer Objects) - Application Services (serviços

de aplicação) - Interfaces (contratos) - Use Cases (casos de uso) #### Infrastructure Layer - Database

(ORM/ODM) - Repositories (implementações) - External Services (serviços externos) - Cache (cache) - Queue

(filas) - Logging (logs) - Providers (injeção de dependência) #### Presentation Layer - Controllers

(controladores) - Middleware (middleware) - Requests (validação) - Resources (transformação) - Routes

(rotas) - Views (templates) #### Shared Layer - Helpers (utilitários) - Traits (traits/mixins) - Constants

(constantes) - Config (configurações) #### Tests - Unit (testes unitários) - Integration (testes de integração) -

Feature (testes funcionais) - E2E (testes end-to-end) #### Docs - API (documentação da API) - Architecture

(ADR - Architecture Decision Records) - Guides (guias do desenvolvedor) #### DevOps - Docker (Dockerfile,

docker-compose) - CI/CD (GitHub Actions, GitLab CI) - Scripts (scripts de automação) ## 2. CÓDIGO

COMPLETO PARA CADA CAMADA Para CADA CAMADA, forneça código completo e funcional com: #### 2.1

Domain Layer - [] Entidades com validação de negócio - [] Value Objects com validação - [] Agregados com

invariantes - [] Eventos de domínio - [] Exceções de domínio - [] Interfaces de repositório - [] Serviços de

domínio ### 2.2 Application Layer - [] Commands com validação - [] Queries com filtros - [] Handlers com lógica de orquestração - [] DTOs com transformação - [] Application Services - [] Casos de uso completos ### 2.3 Infrastructure Layer - [] Implementações de repositório - [] Configuração de banco de dados - [] Serviços externos - [] Cache - [] Filas - [] Logging - [] Injeção de dependência ### 2.4 Presentation Layer - [] Controllers com injeção de dependência - [] Middleware - [] Requests com validação - [] Resources com transformação - [] Rotas ## 3. PADRÕES DE CÓDIGO Crie um guia de padrões de código incluindo: ### 3.1 Nomenclatura - Classes: PascalCase - Methods: camelCase - Variables: camelCase - Constants: UPPER_SNAKE_CASE - Interfaces: [Nome]Interface ou I[Nome] - Abstract Classes: [Nome]Abstract ou Abstract[Nome] ### 3.2 Estrutura de Arquivos - 1 classe por arquivo - Organização por domínio/camada - Namespace/Module organization ### 3.3 Documentação - PHPDoc/JavaDoc/TSDoc - README.md - CONTRIBUTING.md - CHANGELOG.md ## 4. TESTES Crie uma suíte completa de testes: ### 4.1 Testes Unitários (Cobertura > 80%) - Testes de entidades - Testes de value objects - Testes de serviços - Testes de handlers - Mock de dependências ### 4.2 Testes de Integração - Testes de repositórios - Testes de API - Testes de banco de dados - Testes de serviços externos ### 4.3 Testes Funcionais - Testes de endpoints - Testes de autenticação - Testes de autorização - Testes de fluxos completos ## 5. DOCUMENTAÇÃO Crie documentação completa: ### 5.1 README.md - Descrição do projeto - Tecnologias - Pré-requisitos - Instalação - Configuração - Uso - Testes - Deploy - Contribuição - Licença ### 5.2 API Documentation - Endpoints - Request/Response - Authentication - Error codes - Examples ### 5.3 Architecture Decision Records - Contexto - Decisão - Consequências - Alternativas ## 6. CONFIGURAÇÕES DE AMBIENTE ### 6.1 Docker - Dockerfile (multistage) - docker-compose.yml - .dockerignore ### 6.2 CI/CD - GitHub Actions workflows - Deploy scripts - Test automation ### 6.3 Environment Variables - .env.example - Config files ## 7. EXEMPLOS PRÁTICOS CRIE UM CRUD COMPLETO DE [ENTIDADE PRINCIPAL] DEMONSTRANDO: ### 7.1 Criar (Create) - Request validation - Domain entity creation - Repository save - Response transformation ### 7.2 Ler (Read) - Query with filters - Repository find - Response with pagination ### 7.3 Atualizar (Update) - Request validation - Domain entity update - Repository update - Response transformation ### 7.4 Deletar (Delete) - Domain validation - Repository delete - Response ## 8. CHECKLIST DE QUALIDADE Crie um checklist completo para garantir a qualidade do projeto: - [] Arquitetura definida - [] SOLID aplicado - [] Testes implementados - [] Documentação completa - [] CI/CD configurado - [] Docker configurado - [] Security implementado - [] Performance otimizada - [] Code review setup - [] Monitoring configurado - [] Logging implementado - [] Backup strategy ## 9. INSTRUÇÕES FINAIS O código deve ser: - ☒ Funcional e executável - ☒ Bem comentado - ☒ Seguindo padrões de nomenclatura - ☒ Com testes - ☒ Documentado A estrutura deve ser: - ☒ Modular - ☒ Escalável - ☒ Mantível - ☒ Testável - ☒ Independente de tecnologia LINGUAGEM: [PHP, Node.js, Python, Java, etc] FRAMEWORK: [Laravel, Express, FastAPI, Spring Boot, etc] PROJETO: [E-commerce, API REST, Microserviço, etc] ENTIDADE PRINCIPAL: [Product, User, Order, etc] FORNEÇA O CÓDIGO COMPLETO, ORGANIZADO E PRONTO PARA USO. 📄 COMO USAR ESTE PROMPT 1. Substitua os Placeholders LINGUAGEM: [PHP, Node.js, Python, Java, etc] FRAMEWORK: [Laravel, Express, FastAPI, Spring Boot, etc] PROJETO: [E-commerce, API REST, Microserviço, etc] ENTIDADE PRINCIPAL: [Product, User, Order, etc] 2. Exemplo de Prompt Preenchido LINGUAGEM: PHP FRAMEWORK: Laravel PROJETO: E-commerce ENTIDADE PRINCIPAL: Product 3. Exemplo de Prompt para Node.js LINGUAGEM: Node.js FRAMEWORK: Express + TypeScript PROJETO: API REST ENTIDADE PRINCIPAL: User 4. Exemplo de Prompt para Python LINGUAGEM: Python FRAMEWORK: FastAPI PROJETO: Microserviço ENTIDADE PRINCIPAL: Order 📦 VERSÕES DO PROMPT POR LINGUAGEM 🟢 Versão PHP/Laravel 🟡 Versão Node.js/Express LINGUAGEM: TypeScript 5.0 FRAMEWORK: Express.js + TypeORM PROJETO: API REST ENTIDADE PRINCIPAL: User Use: - TypeScript strict mode - TypeORM ou Prisma - Jest para testes - ESLint + Prettier - Winston para logs - Bull para filas - Redis para cache Versão Python/FastAPI LINGUAGEM: Python 3.11 FRAMEWORK: FastAPI + SQLAlchemy PROJETO: Microserviço ENTIDADE PRINCIPAL: Order Use: -

Python 3.11 com type hints - FastAPI com OpenAPI - SQLAlchemy 2.0 - Pytest para testes - Black + Ruff para code style - Celery para filas - Redis para cache - Alembic para migrações 🟡 Versão Java/Spring Boot

LINGUAGEM: Java 17 FRAMEWORK: Spring Boot 3.0 + JPA PROJETO: Sistema de Vendas ENTIDADE

PRINCIPAL: Customer Use: - Java 17 com records - Spring Boot 3.0 - Spring Data JPA - JUnit 5 + Mockito -

Maven/Gradle - Lombok - Flyway para migrações - Spring Cloud para microserviços 🔧 PROMPT PARA

GERAÇÃO DE MÓDULOS ESPECÍFICOS Módulo de Autenticação BASEADO NA ARQUITETURA DEFINIDA,

CRIE UM MÓDULO COMPLETO DE AUTENTICAÇÃO: 1. Domain Layer - Entity: User - Value Objects: Email,

Password, Token - Interfaces: UserRepositoryInterface, AuthServiceInterface - Events: UserRegistered,

UserLoggedIn 2. Application Layer - Commands: RegisterUserCommand, LoginUserCommand - Handlers:

RegisterUserHandler, LoginUserHandler - DTOs: RegisterUserDTO, LoginUserDTO, AuthResponseDTO 3.

Infrastructure Layer - Repository: UserRepository (JWT/Session) - Services: JWTService, PasswordHasher -

Middleware: AuthMiddleware, RoleMiddleware 4. Presentation Layer - Controllers: AuthController -

Requests: RegisterRequest, LoginRequest - Resources: UserResource, TokenResource 5. Tests - Unit:

AuthServiceTest - Integration: AuthControllerTest - Feature: AuthenticationFlowTest FORNEÇA CÓDIGO

COMPLETO E FUNCIONAL. Módulo de Pagamentos BASEADO NA ARQUITETURA DEFINIDA, CRIE UM

MÓDULO COMPLETO DE PAGAMENTO: 1. Domain Layer - Entities: Payment, Transaction - Value Objects:

Money, PaymentStatus - Interfaces: PaymentGatewayInterface, TransactionRepositoryInterface - Events:

PaymentProcessed, PaymentFailed 2. Application Layer - Commands: ProcessPaymentCommand,

RefundPaymentCommand - Handlers: ProcessPaymentHandler, RefundPaymentHandler - DTOs:

PaymentDTO, TransactionDTO, PaymentResponseDTO 3. Infrastructure Layer - Gateway:

MercadoPagoGateway, PayPalGateway, StripeGateway - Repository: TransactionRepository - Services:

PaymentFactory, WebhookHandler 4. Presentation Layer - Controllers: PaymentController,

WebhookController - Requests: PaymentRequest, WebhookRequest - Resources: PaymentResource 5. Tests -

Unit: PaymentServiceTest - Integration: PaymentGatewayTest - Feature: PaymentFlowTest FORNEÇA

CÓDIGO COMPLETO E FUNCIONAL. 🚀 PROMPT PARA GERAR PROJETO COMPLETO CRIE UM PROJETO

COMPLETO DE [TIPO DE PROJETO] USANDO: LINGUAGEM: [PHP/Node.js/Python/Java] FRAMEWORK:

[Laravel/Express/FastAPI/Spring Boot] ARQUITETURA: Hexagonal + DDD + CQRS TESTES: TDD (Test-Driven

Development) DEVOPS: Docker + CI/CD REQUISITOS FUNCIONAIS: 1. [REQUISITO 1] 2. [REQUISITO 2] 3.

[REQUISITO 3] REQUISITOS TÉCNICOS: 1. Autenticação JWT 2. CRUD completo 3. Cache com Redis 4. Filas

com [RabbitMQ/Redis] 5. Logs estruturados 6. Monitoramento 7. API RESTful 8. Documentação OpenAPI

ENTREGÁVEIS: 1. Código fonte completo 2. Testes (unitários, integração, E2E) 3. Documentação (API,

arquitetura, setup) 4. Docker files 5. CI/CD pipelines 6. README completo FORNEÇA TUDO PRONTO PARA

PRODUÇÃO. 📝 COMO USAR ESTES PROMPTS Passo 1: Escolha o Prompt Adequado Use o prompt principal

para projetos completos Use prompts específicos para módulos Use prompts por linguagem para projetos

específicos Passo 2: Substitua os Placeholders Substitua [NOME DO PROJETO] pelo nome real Substitua

[ENTIDADE PRINCIPAL] pela entidade principal Substitua [TIPO DE PROJETO] pelo tipo de projeto Passo 3:

Adicione Detalhes Específicos Adicione requisitos funcionais específicos Adicione regras de negócio Adicione

integrações com sistemas externos Passo 4: Execute o Prompt Cole o prompt em qualquer assistente IA

(ChatGPT, Claude, etc.) Revise o código gerado Adapte conforme necessário 🎯 EXEMPLO DE RESULTADO

ESPERADO Estrutura de Pastas Gerada 📁 meu-ecommerce/ | — 📁 src/ | | — 📁 Domain/ | | — 📁

Entities/ | | | — Product.php | | | — 📁 ValueObjects/ | | | — Money.php | | | — Stock.php | |

| — 📁 Repositories/ | | | — ProductRepositoryInterface.php | | | — 📁 Events/ | | | —

ProductCreated.php | — 📁 Application/ | | | — 📁 Commands/ | | | —

CreateProductCommand.php | | | — 📁 Handlers/ | | | — CreateProductHandler.php | | | — 📁

DTOs/ | | | — ProductDTO.php | — 📁 Infrastructure/ | | | — 📁 Repositories/ | | | —

ProductRepository.php | | | — 📁 Database/ | | | — Migrations/ | — 📁 Presentation/ | | — 📁

```

Controllers/ | | └─ ProductController.php | └─ Requests/ | | └─ CreateProductRequest.php |
└─ Resources/ | └─ ProductResource.php └─ tests/ | └─ Unit/ | | └─ ProductTest.php |
└─ Integration/ | | └─ ProductRepositoryTest.php | └─ Feature/ | └─
ProductControllerTest.php └─ docs/ | └─ api/ | | └─ openapi.yaml | └─ architecture/ |
└─ adr-001.md └─ docker/ | └─ Dockerfile | └─ docker-compose.yml └─ .github/ | └─
workflows/ | └─ tests.yml | └─ deploy.yml └─ README.md └─ .env.example └─ Makefile

```

💡 DICAS PARA MELHORES RESULTADOS

1. Seja Específico Quanto mais detalhes, melhor o resultado Inclua requisitos de negócio Descreva integrações externas
2. Use Exemplos Forneça exemplos de como deve funcionar Inclua casos de uso específicos Descreva fluxos completos
3. Peça Iterações Peça para refatorar partes específicas Peça para adicionar funcionalidades Peça para otimizar performance
4. Revise e Adapte Revise o código gerado Adapte para seu contexto Teste e valide

📖 RECURSOS ADICIONAIS

Templates de Prompt por Caso de Uso

API REST Completa text CRIE UMA API REST COMPLETA COM:

- Autenticação JWT
- CRUD de [entidade]
- Paginação e filtros
- Upload de arquivos
- Email notifications
- Rate limiting
- CORS configurado
- Logs estruturados
- Documentação OpenAPI
- Testes completos

Microserviço text CRIE UM MICROSERVIÇO COMPLETO PARA [função]:

- Comunicação síncrona (REST/gRPC)
- Comunicação assíncrona (RabbitMQ/Kafka)
- Service Discovery
- Config Server
- API Gateway
- Circuit Breaker
- Distributed Tracing
- Health Checks
- Metrics
- Docker/Kubernetes

Sistema de E-commerce text CRIE UM E-COMMERCE COMPLETO:

- Catálogo de produtos
- Carrinho de compras
- Checkout com múltiplos pagamentos
- Gestão de pedidos
- Gestão de usuários
- Wishlist
- Avaliações
- Sistema de frete
- Notificações
- Admin panel
- Relatórios
- SEO
- Analytics

🎉 CONCLUSÃO Com estes prompts profissionais, você pode:

- ✅ Gerar projetos completos em qualquer linguagem
- ✅ Aplicar arquiteturas modernas (Hexagonal, Clean)
- ✅ Seguir padrões de mercado (DDD, SOLID, TDD)
- ✅ Criar código testável e manutenível
- ✅ Automatizar setup de projetos
- ✅ Padronizar desenvolvimento em equipe

Use estes prompts como seu assistente pessoal de arquitetura de software! 🚀